UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Hostel Complaint Management System

Submitted by:

Team Members

NAME	SRN
Anvika Belliappa	PES1UG23CS091
Ayush K Gowda	PES1UG23CS124
Atul Krishna	PES1UG23CS121
Ananya Reddy	PES1UG23CS070

Semester VI - Section: C

Prof. Vikram Dunga

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

Problem Statement:

The existing hostel complaint handling process in many institutions is largely manual, unstructured, and inefficient. Students often face delays in registering complaints, lack of transparency in tracking their status, and poor communication with hostel authorities. This leads to unresolved issues, dissatisfaction among residents, and difficulty in managing complaints systematically.

To address these challenges, a centralized Hostel Complaint Management System is proposed. The system aims to digitize the complaint submission, tracking, and resolution process, ensuring transparency, accountability, and timely action by the concerned authorities.

Key Features:

1. User Authentication & Role-Based Access

- Secure login system for different users:
  - Students
  - Staff
  - Admin
- Role-based dashboards and permissions

2. Complaint Registration

- Students can easily submit complaints
- Categorization of complaints (e.g., maintenance, cleanliness, electrical issues) ● Ability to add descriptions and details

3. Complaint Tracking System

- Real-time tracking of complaint status
- Status updates such as: ○ Pending
  - In Progress
  - Resolved

4. Admin Dashboard

- View all complaints in the system
- Assign complaints to staff members
- Monitor resolution progress

5. Staff Management

- Staff can view assigned complaints
- Update complaint status after action
- Maintain accountability

6. Database Integration

- Centralized storage using MySQL

- Efficient data handling and retrieval

7. Responsive Web Interface

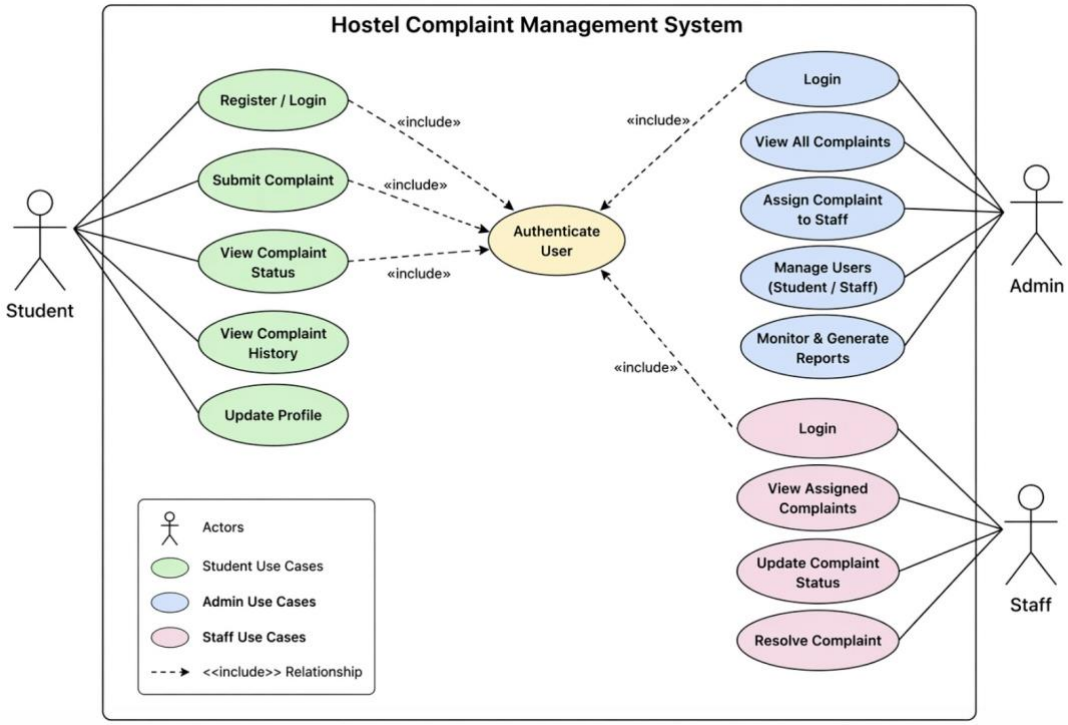
- User-friendly interface using JSP
- Easy navigation for all users

8. Complaint History & Records

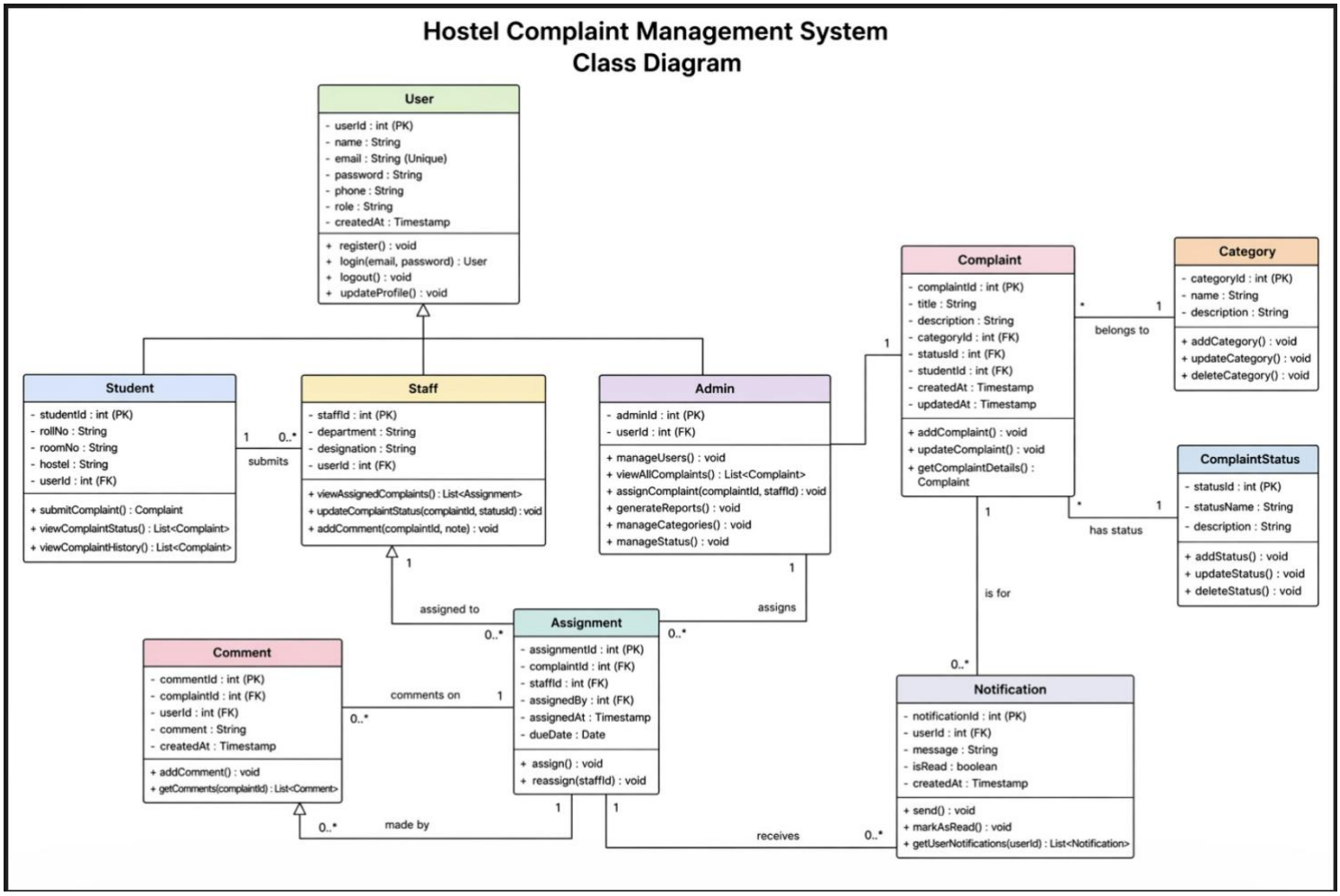
- Maintain past complaint records
- Helps in analysis and future improvements

Models:

Use Case Diagram:

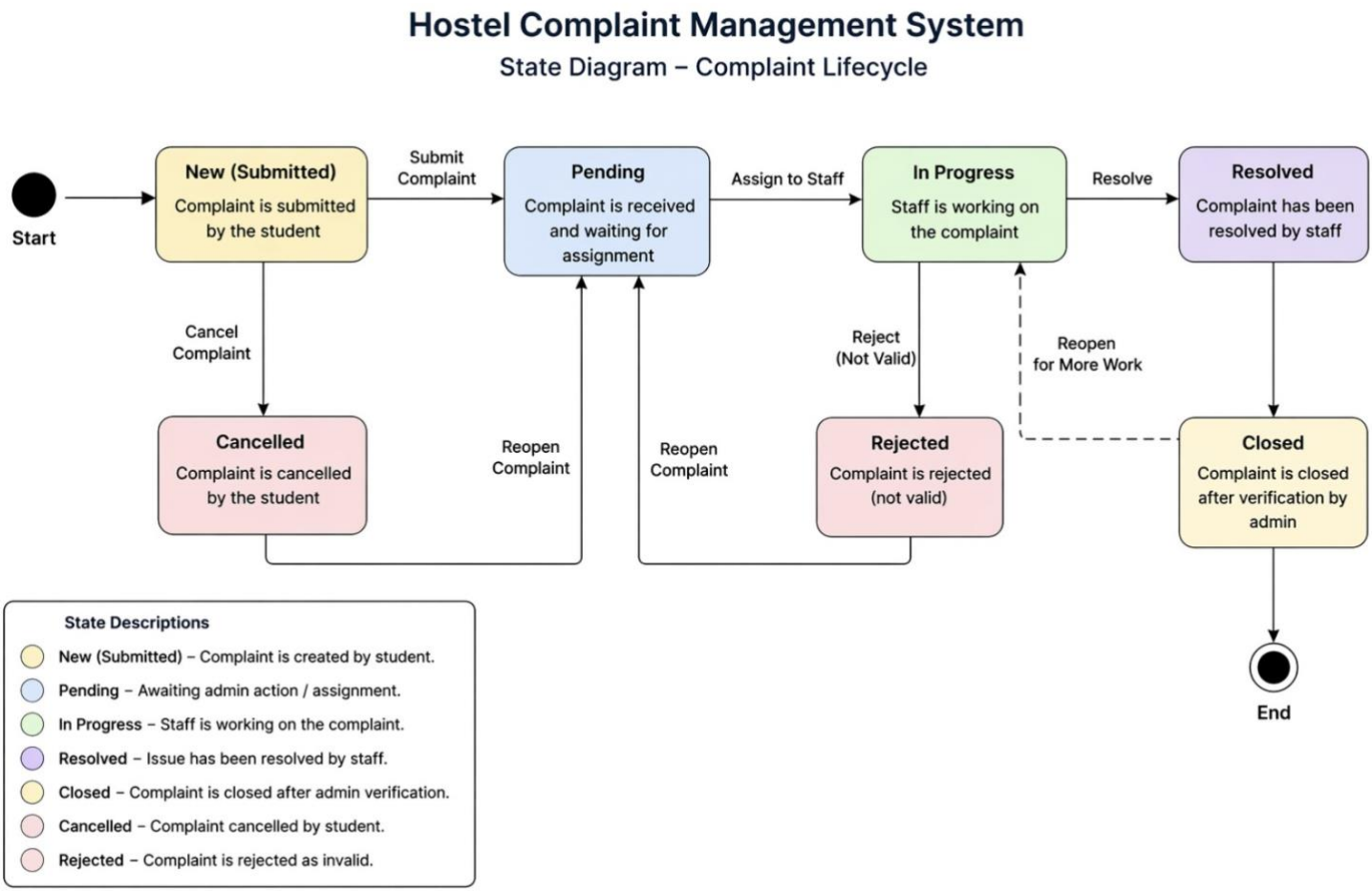


Class Diagram:

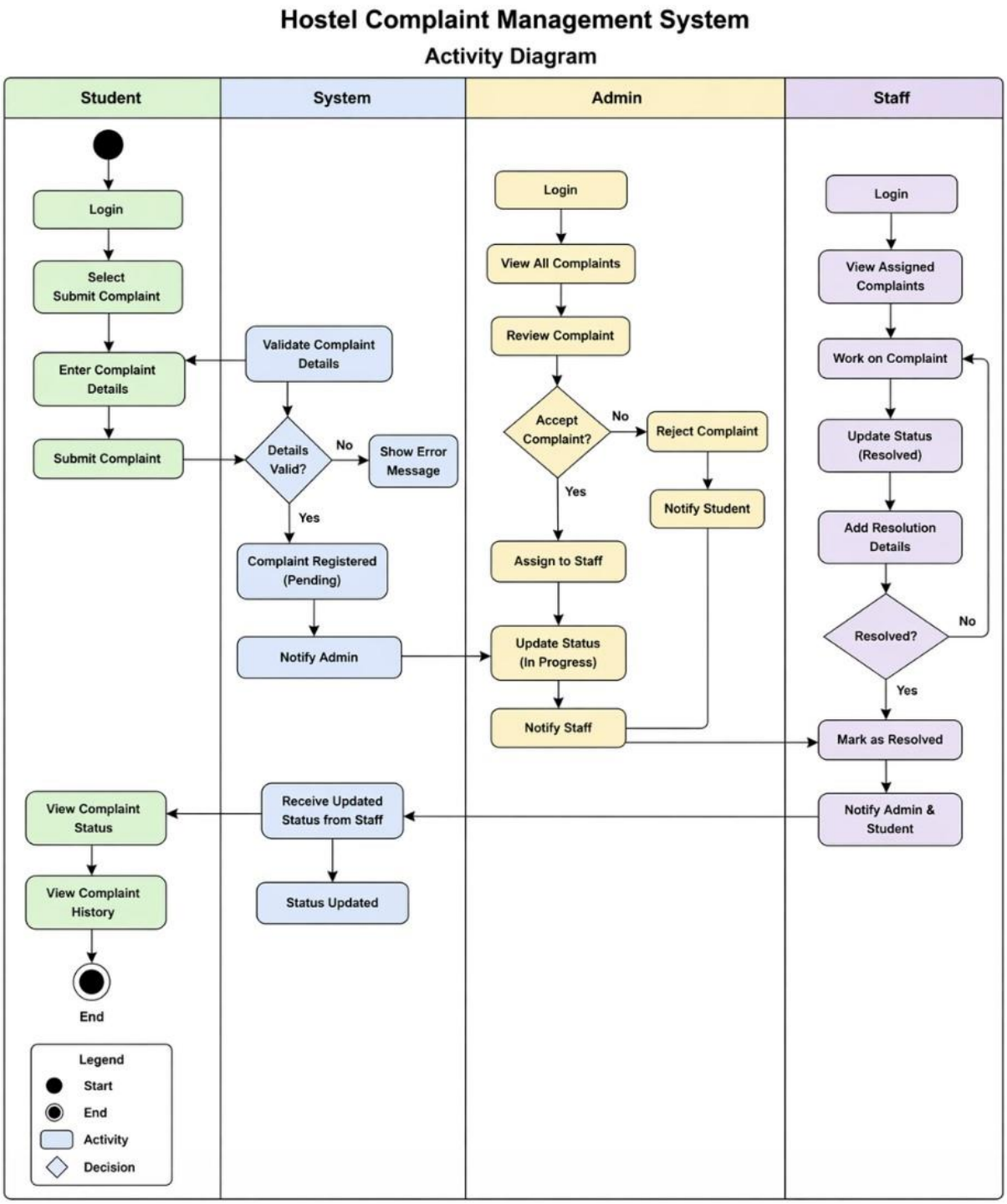


State Diagram:

a



Activity Diagrams:



Design Principles, and Design Patterns:

MVC Architecture used? Yes

Design Principles

The project follows several important software design principles to ensure maintainability, scalability, and clarity:

SOLID Principles

- **Single Responsibility Principle (SRP):**
  Each class performs a single function. For example, DAO classes handle database operations, Servlets handle request processing, and JSP pages handle the user interface.
- **Open/Closed Principle (OCP):**
  The system is designed to allow extension without modifying existing code. New features (e.g., additional complaint types or roles) can be added without altering core classes.
- **Liskov Substitution Principle (LSP):**
  Subclasses or role-based implementations (Admin, Staff, Student) can be used interchangeably without affecting system behavior.
- **Interface Segregation Principle (ISP):**
  Classes use only the methods relevant to them, avoiding unnecessary dependencies and improving modularity.
- **Dependency Inversion Principle (DIP):**
  High-level modules (Servlets) depend on abstractions (DAO interfaces) rather than concrete implementations, reducing coupling.

GRASP Principles

- **Controller:**
  Servlets act as controllers, handling incoming user requests and directing them to appropriate business logic.
- **Information Expert:**
  DAO classes manage database-related operations as they have the required data and logic.
- **Low Coupling:**
  Components such as JSP, Servlets, and DAO classes are loosely coupled, enabling easier maintenance and scalability.
- **High Cohesion:**
  Each class is focused on a specific task, improving clarity and reusability.
- **Creator:**
  Objects are created by classes that have the necessary information (e.g., Servlets creating Complaint objects).

Design Patterns

The following design patterns are implemented in the project:

- **Singleton Pattern:**

  The `DBConnection` class ensures that only one instance of the database connection exists throughout the application, improving resource efficiency.
- **Factory Pattern (implicit usage):**

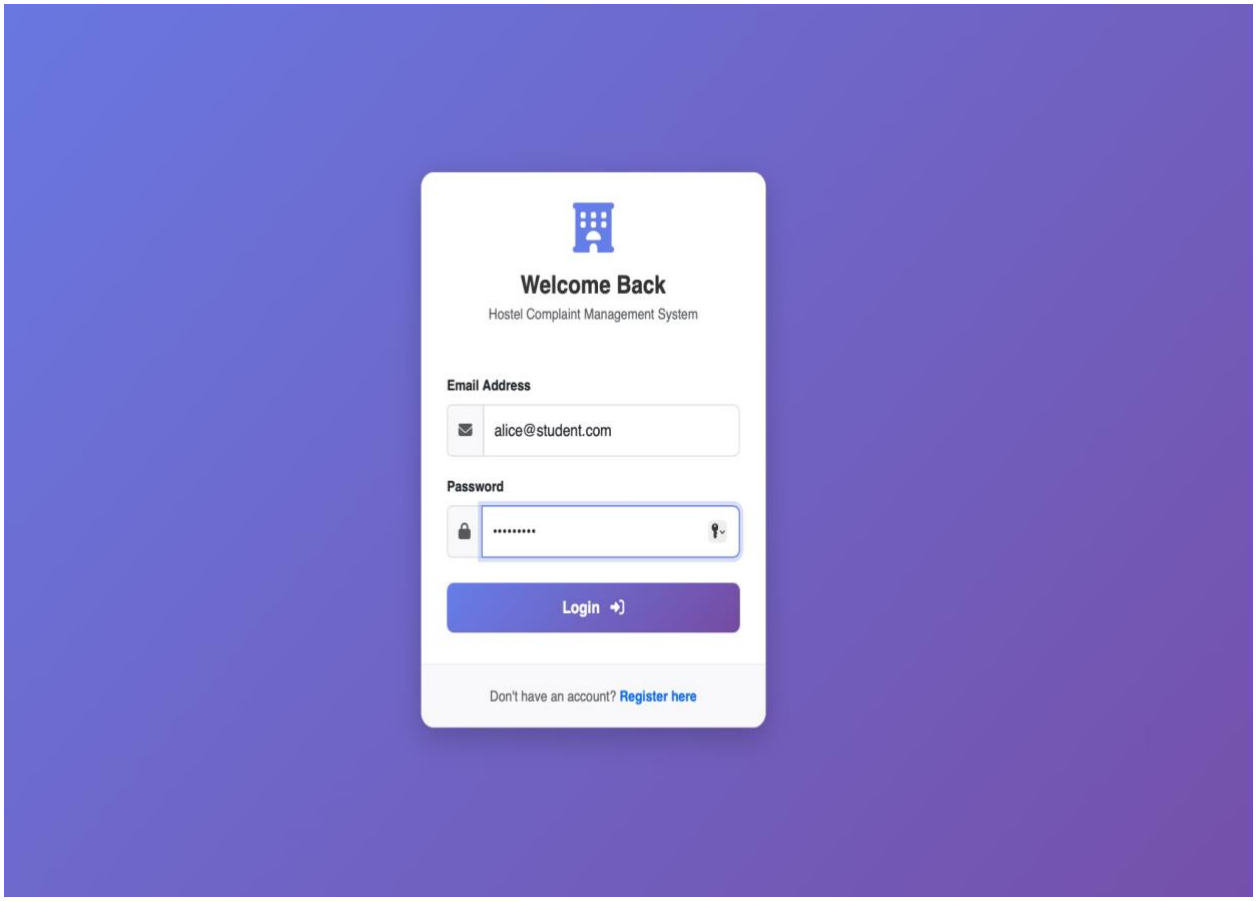
Object creation (e.g., creating user or complaint objects) is handled in a structured way, separating creation logic from usage.

- **MVC (Model-View-Controller) Pattern:**
  The system is structured using MVC architecture, where JSP handles the view, Servlets act as controllers, and DAO classes represent the model. This improves code organization and separation of concerns.
- **DAO (Data Access Object) Pattern:**
  DAO classes such as `UserDAO` and `ComplaintDAO` are used to handle all database interactions. This abstracts database logic from business logic and improves maintainability.
- **Singleton Pattern:**
  The `DBConnection` class follows the Singleton pattern to ensure only one database connection instance is used throughout the application, reducing resource usage.
- **Front Controller Pattern:**
  Servlets act as centralized controllers that handle incoming requests and route them to appropriate components, ensuring consistent request handling.

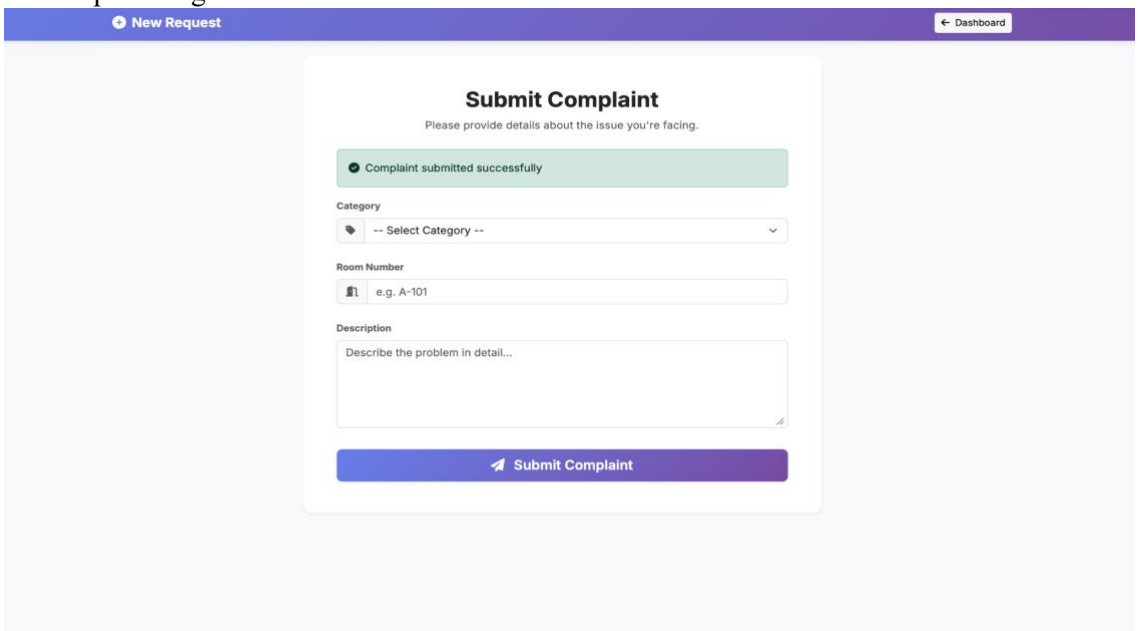
Screenshots

UI:

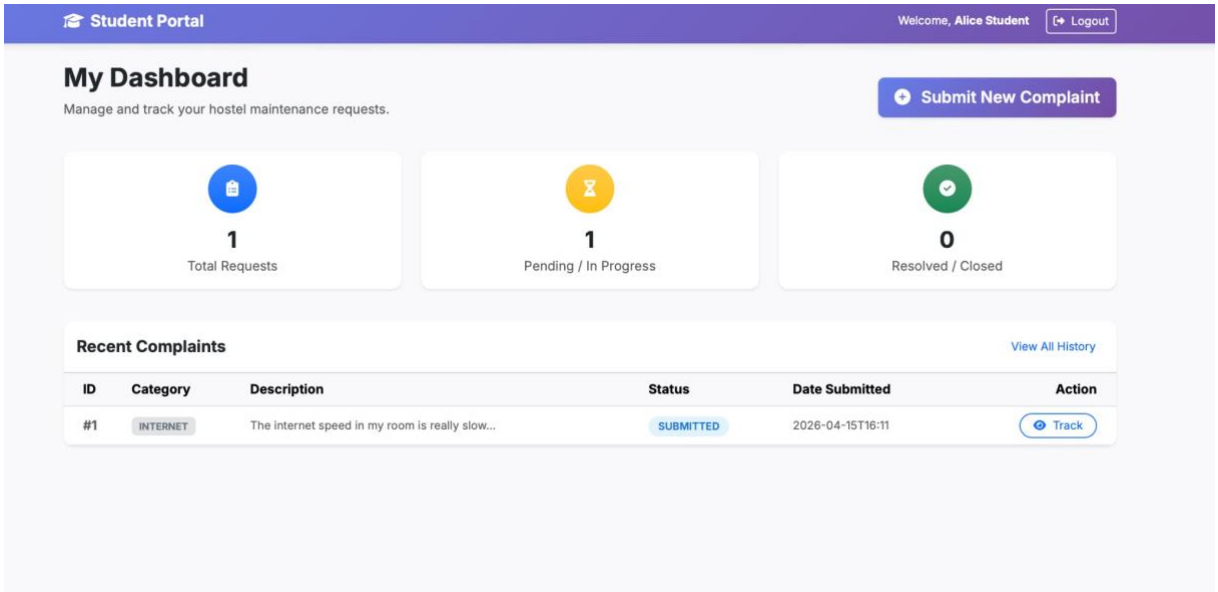
Login page



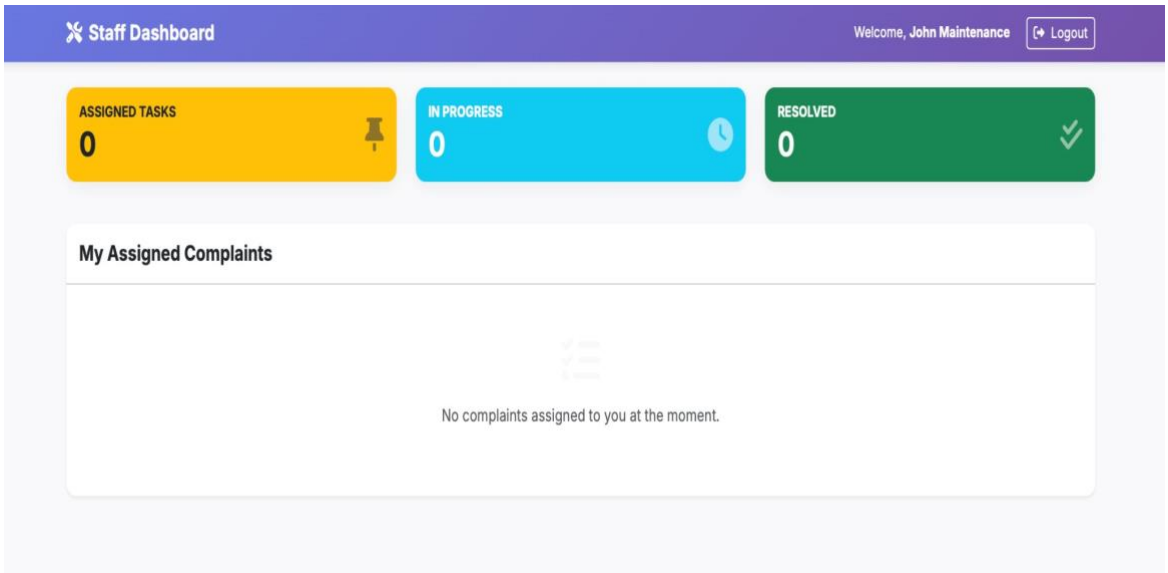
Complaint registration



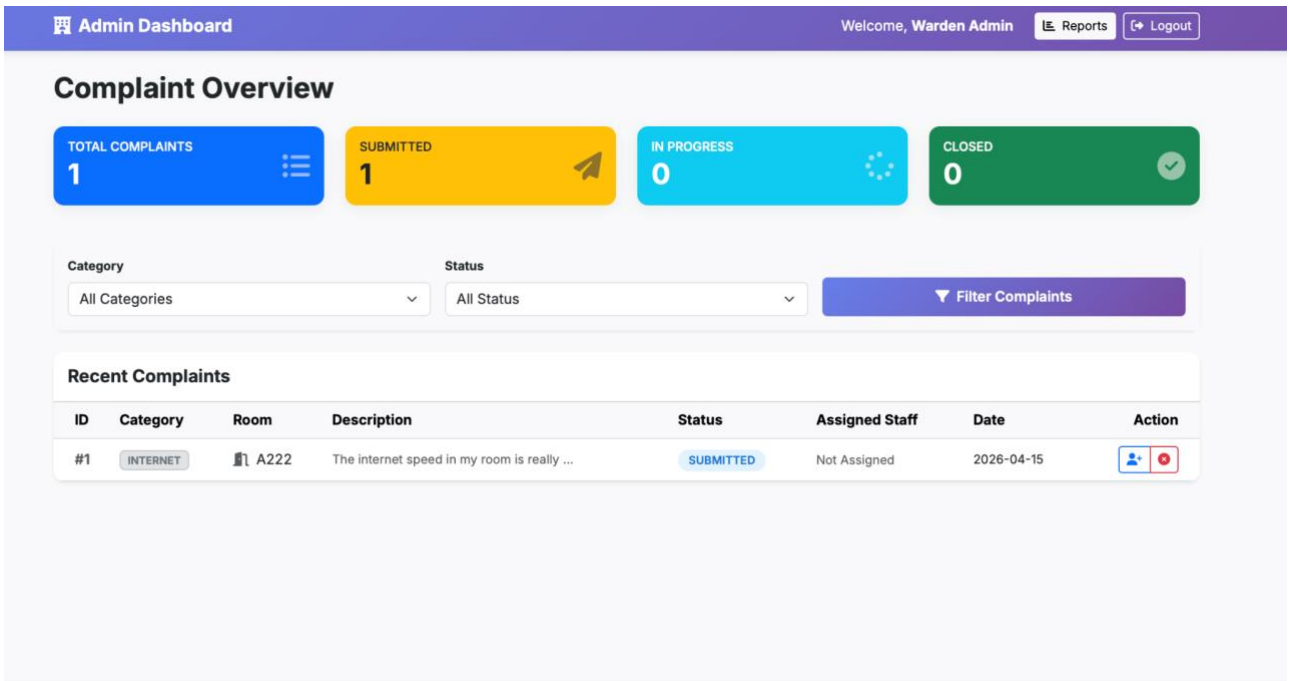
Student Dashboard



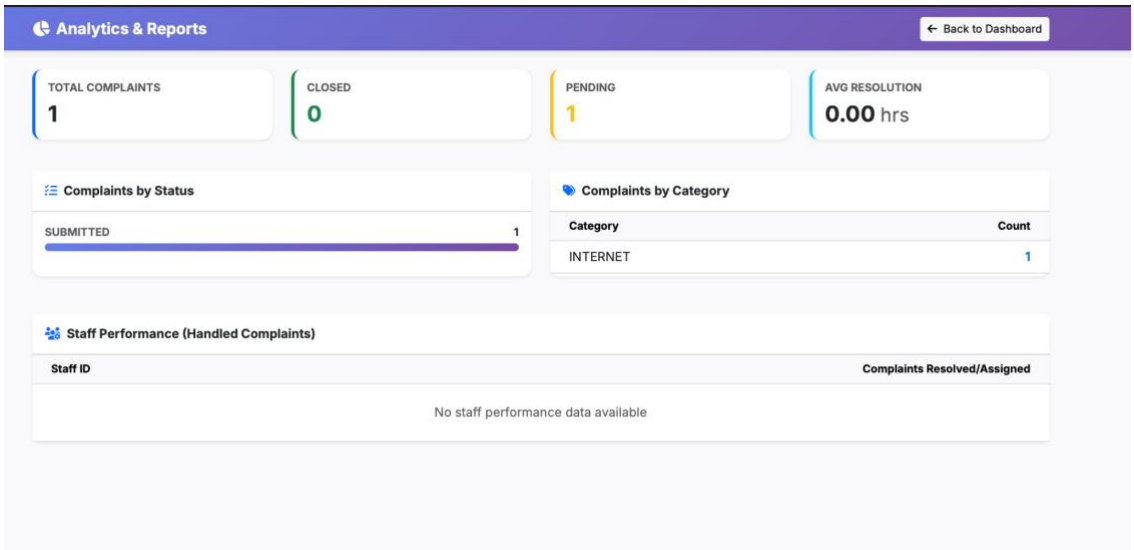
Staff Dashboard



Admin Dashboard



Analytics and Report



Individual contributions of the team members:

Name	Module worked on
Anvika Belliappa	Designed and implemented user authentication system including login and registration using JSP and Servlets
Ayush K Gowda	Developed complaint handling features including submission, tracking, and history management
Atul Krishna	Implemented admin functionalities such as complaint assignment, user management, and monitoring
Ananya Reddy	Managed database integration including MySQL setup, DAO implementation, and backend connectivity